

Admin-Portal

Parity Programme

Every form, field and state the Castilla admin portal keeps about an application — and what the business owner’s app must show, collect or act on so the two halves describe the same permit.

Scope: the Flutter app’s front end only. Nothing here changes the admin portal, the backend, or the contract repo. Where a gap cannot be closed without a server field, the TAB says so and records the hand-off rather than inventing one.

ISSUED	MOBILE LINE	ADMIN LINE SWEPT	TABS	GAPS
25 August 2026	eBPCO-Mobile-App · f603f53	Upupapp/eBPCO-Web · e3cd7c3	16	20 (of 21 raised)

1 · Master Command

THE COMMAND

Bring the eBPCO mobile app to **parity with the admin portal on everything the business owner is a party to**. The admin decides; the applicant must be able to see what was decided, act on what is asked of them, and hold the documents that result.

Work TAB by TAB in order. Each TAB is complete when its acceptance criteria pass, `flutter analyze` is clean, and the whole suite is green. Record every unfinished item with its reason.

What parity does and does not mean

The admin portal is a *case-management* tool. Much of it is rightly invisible to an applicant: user roles, system logs, payroll, tenants, the evaluation queue, the KPI dashboards, and every screen where staff record a decision. None of that belongs on the applicant's phone.

Parity here means something narrower and more demanding: **wherever the admin records a fact about the applicant's own application, the applicant can see that fact**. Wherever the admin asks the applicant for something, the app can supply it. Wherever the admin issues something — an assessment, a receipt, a permit — the applicant can hold it.

The failure this programme exists to prevent. An applicant whose payment was *rejected*, or whose land title was marked *Revision Required*, currently has no screen that can tell them so. The admin has recorded a precise reason; the app has nowhere to put it. The applicant waits, and the 3/7/20-day clock under RA 11032 keeps running.

How the gaps were found

The vendor monorepo `Upupapp/eBPCO-Web` was fetched and fast-forwarded nine commits to `e3cd7c3` (25 Aug), then its Angular admin — `EBPCO WEB ADMIN/E-BPCO-Software-main` — was read domain model by domain model and compared against the Flutter line at `f603f53`. Each gap below cites the file it came from on both sides.

Two divergent lines. The vendor's own mobile copy (`ebpc0-web/ebpc0-mobile`) is not this app, and in the nine commits just pulled it grew `fsic_permit` and `zoning_permit` wizards of its own. That is corroboration that TABs 03–05 are real gaps, and a reference to read — **not** code to copy: the two lines have diverged since 19 August and this one is ahead on almost everything else.

Table of contents

§2	What the admin keeps, and what the app shows
§3	Gap register G-01 ... G-21
TAB 00	Shared vocabulary & parity guard
TAB 01	Requirements catalog on mobile
TAB 02	Per-document review status
TAB 03	Zoning / Locational Clearance
TAB 04	FSEC for Building Permit
TAB 05	FSIC for Occupancy Permit
TAB 06	Partial payment & balance
TAB 07	Official Receipt & collecting agency

TAB 08	Assessment versions
TAB 09	Release, claim & validity
TAB 10	Official forms & checklists
TAB 11	Contact verification
TAB 12	Citizen's Charter coverage
TAB 13	Notification alignment
TAB 14	Renewal & amendment
TAB 15	Closing certification
§4	Cross-cutting requirements X1–X5

2 · What the admin keeps, and what the app shows

Already aligned — do not disturb

Three vocabularies match across both lines and should be treated as settled. They are listed so no TAB “fixes” them.

Concept	Admin	Mobile	Verdict
Application lifecycle	19 states, <code>status.model.ts</code>	19 states, <code>lifecycle_status.dart</code>	Aligned, incl. the applicant-label projection
Evaluation stages	Initial · Zoning · Fire Safety · OBO · Final Approval	Same five, same order	Aligned
Application action	New · Renewal · Amendment	<code>ApplicationType</code>	Type aligned; the flows are not — see TAB 14

The permit catalog

The admin defines **19** permit types in one place (`ALL_PERMIT_TYPES` , `permit.model.ts`) and guards them with `isValidPermitType` . The app files **16**.

Permit type (admin’s exact string)	Requirement docs	Validity	Mobile
Building Permit – New Construction	22	12 mo	Yes
Building Permit – Renovation / Alteration	9	12 mo	Yes
Building Permit – Addition / Extension	9	12 mo	Yes
Demolition Permit	9	6 mo	Yes
Zoning / Locational Clearance	16	12 mo	MISSING
Architectural Permit	7	12 mo	Yes
Civil / Structural Permit	8	12 mo	Yes
Electrical Permit	7	12 mo	Yes
Mechanical Permit	7	12 mo	Yes
Sanitary Permit	7	12 mo	Yes — but named “Sanitary / Plumbing”
Plumbing Permit	7	12 mo	Yes
Electronics Permit	7	12 mo	Yes
Interior Design Permit	7	12 mo	Yes
Fencing Permit	6	6 mo	Yes
Sign Permit	7	12 mo	Yes
Excavation Permit	8	6 mo	Yes
FSEC for Building Permit (BFP)	9	12 mo	MISSING
Certificate of Occupancy	9	no expiry	Yes
FSIC for Occupancy Permit (BFP)	10	12 mo	MISSING

Counting note. Document counts are the admin’s own `REQUIREMENTS_CATALOG` entries: five common documents plus the type’s plan/professional/extra documents, except New Construction and Zoning, which replace the common set with a

transcribed official Castilla checklist. A first pass at this table undercounted several types because the source splits specs across shared constants; the figures above are from a brace-matched parse and were spot-checked against the file.

The asymmetry that matters most

The app already collects a great deal — between 3 and 17 upload slots per wizard. What it does not have is the *catalog*: the admin's per-permit statement of which documents are required, which office reviews each one, how long the permit is valid, what inspection follows, and which law or Castilla form the requirement comes from. The app's lists are hardcoded inside sixteen wizards, so they can drift from the admin's without anything failing.

The clearest instance: Building Permit – New Construction carries **6 upload slots on mobile** against a **22-line official checklist** in the admin. That is not necessarily sixteen missing uploads — some mobile slots aggregate several checklist lines — but nothing in either codebase can currently tell you which.

3 · Gap register

Twenty gaps that stand, plus one withdrawn on inspection, each traced to the admin file that defines it and the mobile file that would carry it. Severity is the applicant's exposure, not the size of the work.

ID	Gap	Admin source	Sev	TAB
G-01	Per-document review status absent — 8 admin states, mobile's <code>DocumentModel</code> has none	<code>document.model.ts</code>	P0	02
G-02	Document rejection <code>remarks</code> never reach the applicant	<code>ApplicationDocument.remarks</code>	P0	02
G-03	Payment rejection reason has nowhere to display	<code>PaymentTransaction.rejectionReason</code>	P0	06
G-04	"Partially Paid" unrepresentable — mobile has 4 payment states to the admin's 5	<code>status.model.ts</code>	P0	06
G-05	Official Receipt (number, date, issuer) not modelled	<code>PaymentTransaction.orNumber</code>	P0	07
G-06	Zoning / Locational Clearance cannot be filed	<code>ALL_PERMIT_TYPES</code>	P0	03
G-07	FSEC cannot be filed	<code>ALL_PERMIT_TYPES</code>	P0	04
G-08	FSIC cannot be filed	<code>ALL_PERMIT_TYPES</code>	P0	05
G-09	No requirements catalog — 16 hardcoded lists that can drift	<code>requirements-catalog.ts</code> (894 lines)	P1	01
G-10	Permit validity / expiry absent — mobile models PD 1096 commencement only, and assumes 12 months for every type	<code>validityMonths</code> (6 / 12 / null)	P1	09
G-11	Release method & claimant not modelled — WITHDRAWN, see §3.1. Mobile models release more fully than the admin does on the applicant's side.	—	N/A	—
G-12	Collecting agency (OBO/LGU vs BFP) not distinguished	<code>CollectingAgency</code>	P1	07
G-13	Assessment supersession invisible — a revised assessment looks like the first	<code>AssessmentStatus</code> 'Superseded'	P1	08
G-14	Fee line items not itemised from a versioned assessment	<code>AssessmentLineItem</code>	P1	08
G-15	Official blank forms & checklists not available in-app	<code>permit-form-templates.ts</code>	P1	10
G-16	Contact verification (email link / mobile OTP) absent	<code>ContactVerification</code>	P1	11
G-17	Citizen's Charter covers 5 permit types of 19	<code>REQUIREMENTS_CATALOG</code>	P1	12
G-18	Document issuing office / issue & expiry dates not captured	<code>ApplicationDocument</code>	P2	02
G-19	Payment adjustments (void, reversal, refund, correction) invisible	<code>PaymentAdjustment</code>	P2	07

G-20	Renewal & amendment have no distinct flow despite a shared type	<code>ApplicationAction</code>	P2	14
G-21	"Sanitary Permit" filed as "Sanitary / Plumbing Permit" — string mismatch	<code>ALL_PERMIT_TYPES</code>	P2	00

Why the P0s are P0. Each of G-01 through G-08 is a case where the admin has recorded something the applicant must act on — a rejected document, a rejected payment, a permit type they are legally required to obtain — and the app is structurally incapable of telling them. Under RA 11032 the processing clock does not stop because the applicant was never informed.

3.1 · One gap withdrawn, and why it matters

G-11 was wrong and is withdrawn. It claimed mobile does not model permit release, citing a comment in the admin's own `permit.model.ts` : "the mobile app itself has not implemented a releasing-officer/claimant/release-method model yet (its ApplicationModel only stamps permitNumber + issuedDate on release)".

That comment is true of the *vendor's* mobile copy. It is not true of this line. `ReleaseRecord` in `application_detail.dart` carries status, method, claimant, released-at, *and* claim location, office hours and what to bring — the last three having no counterpart in the admin at all. On the applicant's side this app is ahead, not behind.

The lesson, which this document nearly published as a work item. The two lines diverged on 19 August. A comment in one describing "the mobile app" describes *its* mobile app. Every claim about what this line lacks must be checked against this line's source, not against the other line's commentary about it — including claims found in a file otherwise being used as the authority.

What survives, sharpened: mobile computes `commenceByDate` as issue + 1 year, which is **PD 1096's commencement deadline** — the year in which work must start — and is correct. It is *not* permit validity. The catalog gives validity per type as 6 months, 12 months, or none at all for the Certificate of Occupancy. Mobile has no expiry field and no per-type validity, and the two concepts must not be merged when TAB 09 adds one.

PHASE A

Make the two halves speak the same language

Before any screen is built, the app needs the admin's vocabularies as data rather than as sixteen hardcoded opinions. TABs 00–02 are the foundation every later TAB stands on.

00 Shared vocabulary & parity guard

Closes G-21 · Enables every other TAB · **MOBILE FE**

WHY

The admin defines its permit types, document statuses, payment statuses and assessment statuses once each, and validates against them. The app defines its own, in different files, with different strings. Nothing compares the two, so the first divergence is discovered by an applicant.

BUILD

- A single `lib/core/contract/` module holding the admin's exact vocabularies: `PermitType` (all 19 strings verbatim, including the en-dash in “Building Permit – New Construction”), `DocumentStatus` (8), `PaymentStatus` (5), `AssessmentStatus` (8), `PaymentTransactionStatus` (4), `CollectingAgency`, `ReleaseMethod`, `VerificationStatus`.
- Each with a `fromWire` that **rejects** an unknown value rather than defaulting — the admin does exactly this in `isValidPermitType`.
- Rename the app's “Sanitary / Plumbing Permit” to the admin's “**Sanitary Permit**” everywhere it is filed or displayed, and keep the wizard directory name as-is to avoid a pointless rename.

ACCEPTANCE

- ☐ A test asserts all 19 permit-type strings byte-for-byte against a fixture copied from `ALL_PERMIT_TYPES`.
- ☐ A test asserts each vocabulary's value count and rejects an unknown string.
- ☐ An architecture test fails if any wizard files a permit-type string not in the contract module.
- ☐ No user-visible string says “Sanitary / Plumbing Permit”.

The trap this TAB is designed around. Two same-named enums in different files with different members have already caused a near-miss on this codebase (an `Architectural` status enum reading ‘Issued’ where thirteen others read ‘Approved’). Put the wire vocabulary in one module and make every consumer import it.

01 Requirements catalog on mobile

Closes G-09 · Depends on TAB 00 · MOBILE FE P1

WHY

`requirements-catalog.ts` is 894 lines and is the admin's answer to "what does this permit actually require". It carries, per permit type: the required official form, every document with a required flag and reviewing department, the evaluation sequence, payment and inspection requirements, validity rules and months, the final document issued, release requirements, and cited legal sources with a verification status distinguishing national law from a Castilla form from a pending placeholder.

The app has none of it. Its document lists live inside sixteen wizard step files.

BUILD

- `lib/core/contract/requirements_catalog.dart` — a Dart mirror, one entry per permit type, generated from the admin file rather than retyped.
- Drive each wizard's Required Documents step *from the catalog* instead of hardcoded slots. Where a mobile slot aggregates several catalog lines, make that explicit in the entry rather than silently dropping lines.
- Show, on the pre-flight screen: the required official form, the count of required vs optional documents, the validity period, and what inspection follows.
- Carry `verificationStatus` through: a requirement sourced from a *pending Castilla verification* must not be presented to an applicant as settled law.

ACCEPTANCE

- ☐ A test asserts the Dart catalog has an entry for all 19 types and that document ids match the admin's.
- ☐ Every wizard's upload slots derive from the catalog; an architecture test fails on a hardcoded list.
- ☐ Pre-flight names the official form and the validity period for the chosen permit.
- ☐ Requirements not yet verified against a Castilla source are visibly marked as provisional.

02 Per-document review status

Closes G-01, G-02, G-18 · **MOBILE FE** **P0**

WHY

The admin tracks each submitted document through **Missing** → **Uploaded** → **Submitted** → **Under Review** → **Accepted / Rejected / Revision Required / Expired**, requires **remarks** whenever it lands on Rejected or Revision Required, keeps an append-only **history** of earlier submissions, and records the document's own issuing office, issue date and expiry date.

Mobile's `DocumentModel` is `id · label · fileName · uploadedAt · fileSizeBytes · filePath`. There is no status field at all. The applicant learns a document was rejected only if the office happens to raise a Letter of Instruction about it.

BUILD

- Extend the document model with status, remarks, issuing office, issue and expiry dates, and submission history.
- On the application detail screen, a requirements list showing each document's current status, with rejected and revision-required items surfaced first and their remarks shown in full.
- A **resubmit** action on any document in an unresolved state, which appends to history rather than overwriting.
- A document whose **expiryDate** has passed is shown as Expired before the office marks it so — the applicant can act sooner than the desk.

ACCEPTANCE

- ☐ Each of the 8 statuses renders distinctly, with colour never the only signal.
- ☐ A rejected document shows its remarks; a test asserts remarks are never silently dropped.
- ☐ Resubmission preserves the prior submission in history.
- ☐ The detail screen orders unresolved documents above resolved ones.

PHASE B

The three permits that cannot be filed

Zoning, FSEC and FSIC are in the admin's catalog, have real Castilla and BFP forms bundled, and route to offices other than the OBO. An applicant cannot start any of them from the app.

03 Zoning / Locational Clearance

Closes G-06 · Office MPDO, not OBO · MOBILE FE PO

WHY

Locational clearance is ordinarily a *precondition* of the Building Permit — the common document set for most other types lists it as a required input. The app asks applicants to upload a Locational Clearance it gives them no way to obtain.

WHAT THE ADMIN SPECIFIES

Required form: **Application for Locational Clearance / Certificate of Zoning Compliance (Form FM-MPD-12)**, a real Castilla MPDO form. Sixteen documents, including a notarized letter request to the Zoning Administrator, site development plan, vicinity map, sketch plan, bill of materials, proof of ownership, tax declaration / COT / OCT, current land tax receipt, barangay building clearance, cedula, and — conditionally — DPWH clearance and an ECC. Twelve-month validity. Inspection: ocular site inspection and a project evaluation report by the Zoning Officer.

BUILD

- A wizard following the app's established shape, with steps derived from the catalog entry rather than invented.
- Route it to the MPDO in any office-facing copy — `responsibleDepartmentId` is `zoning`, not `obo`.
- Mark the two conditional documents (DPWH, ECC) as optional, with the "if applicable" condition stated.
- Register it in the permit catalog screen under the correct grouping and in `DraftRegistry`.

ACCEPTANCE

- ☐ The wizard files a permit whose type string is exactly `Zoning / Locational Clearance`.
- ☐ All 16 catalog documents are represented; required and optional are distinguished.
- ☐ The draft appears in the Drafts segment and resumes correctly.
- ☐ Accessibility suites cover it at 100%, 200%, 320dp and the 48dp floor.

04 FSEC for Building Permit (BFP)

Closes G-07 · Office Bureau of Fire Protection · **MOBILE FE** **P0**

WHY

The Fire Safety Evaluation Clearance is required under RA 9514 before a Building Permit issues. The admin bundles the real BFP Castilla Fire Station form and sources the requirement to the BFP's own ARTA citizen's charter. The app has no FSEC flow, and its Fire Safety evaluation stage therefore reports on a clearance the applicant cannot apply for.

BUILD

- A wizard from the catalog's nine-document entry.
- Fees collect to **BFP, not the LGU** — see TAB 07. Do not present an FSEC fee as payable at the OBO cashier.
- Explain the dependency plainly: this clearance is an input to the Building Permit, and its own notice should say so rather than implying it stands alone.

ACCEPTANCE

- ☐ Files a permit typed exactly **FSEC for Building Permit (BFP)** .
- ☐ Any fee shown is attributed to the BFP as collecting agency.
- ☐ The relationship to the Building Permit is stated on both screens.
- ☐ Accessibility and clipping suites cover it.

05 FSIC for Occupancy Permit (BFP)

Closes G-08 · Office Bureau of Fire Protection · **MOBILE FE** **P0**

WHY

The Fire Safety Inspection Certificate is the occupancy-side counterpart and is a precondition of the Certificate of Occupancy. Ten documents in the admin's catalog, real BFP form bundled. The app's Certificate of Occupancy wizard asks for an FSIC it cannot produce.

BUILD

- A wizard from the ten-document catalog entry.
- Link it from the Certificate of Occupancy wizard at the point where the FSIC is requested, so an applicant who lacks one can start it without leaving the flow and losing their draft.
- BFP collecting agency, as TAB 04.

ACCEPTANCE

- ☐ Files a permit typed exactly **FSIC for Occupancy Permit (BFP)** .
- ☐ Starting an FSIC from inside the Certificate of Occupancy wizard preserves the occupancy draft.
- ☐ Accessibility and clipping suites cover it.

PHASE C

Money the applicant can account for

The admin runs a versioned assessment with line items, partial payment, two collecting agencies, transaction-level verification and official receipts. The app shows a single total and four states.

06 Partial payment, rejection & balance

Closes G-03, G-04 · MOBILE FE P0

WHY

The admin's own comment on `PaymentStatus` is explicit: it extends mobile's four values with **Partially Paid**, because the versioned assessment model "supports genuine partial payment of a staged/legally-prescribed charge, which mobile's coarser 4-value status has no room to represent", and it instructs every consumer to handle it explicitly rather than falling through a default.

Separately, a payment transaction can be **Rejected** with a required `rejectionReason`. Mobile cannot show either.

BUILD

- Add `partiallyPaid` to the payment status, and an amount-paid / balance-due pair alongside the total.
- Render each payment transaction the applicant has made, with its status, reference, date, and — when rejected — the reason in full and an obvious way to submit a corrected payment.
- Never let a rejected transaction count toward the balance; the admin excludes voided transactions from every total and the app must agree.
- Money stays integer centavos throughout. No `double`.

ACCEPTANCE

- ☐ A partially paid assessment shows amount paid, balance due, and does not read as "Paid".
- ☐ A rejected transaction shows its reason and offers a retry path.
- ☐ A test asserts rejected and voided transactions are excluded from the balance.
- ☐ No arithmetic on payment amounts uses floating point.

07 Official Receipt & collecting agency

Closes G-05, G-12, G-19 · **MOBILE FE** **P0**

WHY

The Official Receipt is the applicant's proof of payment to a government office. The admin records `orNumber`, `orDate` and `orIssuedBy` per transaction and is careful to note they are never auto-filled. The app references an OR number in four wizard steps and one notification payload, but its `PaymentAssessmentModel` has no field for one — so there is nowhere for the real value to live.

The admin also splits collection between **OBO/LGU** and **BFP**. An applicant paying an FSIC fee is paying a different office, at a different counter.

BUILD

- Carry the OR triple on the payment transaction and display it prominently once issued; make it copyable.
- Show the collecting agency on every fee, assessment and receipt.
- Surface payment adjustments — void, reversal, refund, correction — with their effect on the balance, rather than silently changing the number.
- Do not fabricate an OR number in any mock or seed path. An absent OR shows as “not yet issued”.

ACCEPTANCE

- ☐ A verified payment shows OR number, date and issuing officer.
- ☐ Every fee line names its collecting agency; BFP fees are never shown as LGU.
- ☐ An adjustment is visible as an adjustment, with the prior balance still legible.
- ☐ A test asserts no code path invents an OR number.

08 Assessment versions & line items

Closes G-13, G-14 · **MOBILE FE** **P1**

WHY

The admin's assessment is versioned and immutable: each line item is a frozen copy of the fee rule as it stood when the assessment was built, and a correction after issuance creates a **new version** and marks the old one **Superseded**. An applicant who was assessed, then re-assessed after a revision, currently sees a single changed number with no indication that it changed or why.

BUILD

- Model the assessment as versioned, with status including Superseded and Voided.
- Show the current assessment's line items with their basis, and offer the superseded version for comparison rather than discarding it.
- State plainly when an assessment has been replaced, and what to pay now.

ACCEPTANCE

- ☐ A superseded assessment is labelled as such and is not payable.
- ☐ Line items show the fee basis captured at assessment time.
- ☐ A reassessment is visible as a change, with the prior version reachable.

The permit in the applicant's hands

Issuance, claim and validity — plus the reference material that should have been available before they started.

09 Permit validity & expiry

Closes G-10 · G-11 withdrawn · MOBILE FE P1

WHY

Every permit type except the Certificate of Occupancy expires — six months or twelve, per the catalog's `validityMonths`. Mobile has no expiry at all. What it does have is `GeneratedPermit.commenceByDate`, computed as issue date plus one year, which is a different thing: PD 1096's requirement that authorised work *commence* within a year or the permit is void.

Two deadlines, not one. Commencement (PD 1096, one year, already built and correct) and validity (per permit type, 6 or 12 months or none) are separate obligations with separate consequences. A Fencing Permit is valid six months but must be commenced within twelve — so its validity expires *first*. Merging them would tell the applicant the wrong date.

BUILD

- Carry `validityMonths` from the TAB 01 catalog onto the issued permit and derive an expiry date from it. Null for Certificate of Occupancy — render “no fixed expiry”, not a blank.
- Show both dates where both apply, each labelled for what it is, and warn as each approaches. Reuse the existing warning-threshold constant rather than adding a second.
- Leave `ReleaseRecord` alone. It already carries more of what an applicant needs than the admin does; the only admin field it lacks is the releasing officer, which is staff-facing.

ACCEPTANCE

- ☐ An issued permit shows its validity period and expiry date, derived from the catalog.
- ☐ A Certificate of Occupancy shows no expiry, because its validity is null.
- ☐ Commencement and expiry are separately labelled and never conflated.
- ☐ A test asserts a 6-month permit expires before its commencement deadline.

10 Official forms & checklists

Closes G-15 · MOBILE FE P1

WHY

The admin bundles the **real** Castilla and BFP forms per permit type — the OBO's trade-permit forms, the Unified Application Form for Building Permit (one form serving all three building sub-types via its Scope of Work checkboxes), the MPDO Locational Clearance form, and the BFP FSEC and FSIC forms — plus a checklist PDF per type. Applicants who want to prepare offline, or who are helping a professional fill the paper form, have no access to any of it from the app.

BUILD

- Bundle the same PDFs and expose them from the pre-flight screen and the permit catalog: “View the official form” and “View the checklist”.
- Where the admin has no genuine Castilla form for a type — Architectural, Interior Design, Sign, Demolition, Certificate of Occupancy — say the template is a reference rather than the official form. Do not present a placeholder as an LGU document.
- Reuse the existing document preview screen; do not add a second viewer.

ACCEPTANCE

- ☐ Every permit type with a bundled form can open it without leaving the app.
- ☐ Reference-only templates are labelled as such.
- ☐ A type with no form shows no broken link.

11 Contact verification

Closes G-16 · MOBILE FE P1

WHY

The admin models an applicant's contact verification with four methods — email verification link, mobile OTP, manual administrator confirmation, and verified-document matching — and four statuses. The app registers an account and verifies nothing, so the office cannot rely on the contact details it will use to send every notice.

BUILD

- Show verification state on the profile, per channel, honestly: an unverified mobile number says Unverified.
- Offer the two applicant-driven methods (email link, mobile OTP) and handle the failure state, which the admin models explicitly.
- Do not gate existing functionality behind verification in this TAB — that is a product decision, recorded not taken.

ACCEPTANCE

- ☐ Profile shows per-channel verification status.
- ☐ A failed verification is distinguishable from an unattempted one.
- ☐ Verification requests survive a failed network call — see X3.

Backend hand-off. Email links and OTP cannot be completed front-end-only. This TAB builds the states, the screens and the honest reporting; the sending and checking is a backend item and must be recorded as one rather than mocked into looking finished.

12 Citizen's Charter coverage

Closes G-17 · MOBILE FE P1

WHY

The app's Citizen's Charter — which the pre-flight screen reads to tell an applicant how many documents they will need — covers **five** permit types: New Construction, Renovation, Addition / Extension, Demolition, and Certificate of Occupancy. The catalog has nineteen. For fourteen permit types the app cannot answer “what will this take”.

BUILD

- Extend the charter to all 19 types, sourced from the TAB 01 catalog so the two cannot disagree.
- Keep the RA 11032 processing-day pledge per type, and the working-day engine already built for it.
- Where a type's requirements are still provisional, the charter says so.

ACCEPTANCE

- ☐ All 19 types have a charter entry; a test asserts the count matches the catalog.
- ☐ Pre-flight gives a document count for every type.
- ☐ Charter requirement counts are derived, never retyped.

13 Notification alignment

Supports G-01...G-05 · **MOBILE FE** **P2**

WHY

The admin generates notifications from domain mutations, so every one references a real application. The app has its own catalog and its own evaluator for conditions it can derive locally. The new states introduced by TABs 02, 06 and 07 — document rejected, payment rejected, OR issued, assessment superseded — need notifications, or the applicant only finds out by opening the app and looking.

BUILD

- Add notification types for each new applicant-actionable state, with stable dedupe keys in the pattern already established.
- Keep the read / resolved distinction: a notice about a rejected document is resolved when the document is resubmitted, not when it is read.
- Respect quiet hours as the existing evaluator does.

ACCEPTANCE

- ☐ Every new state has a notification with a stable dedupe key.
- ☐ Resolving the underlying condition resolves the notice.
- ☐ No notification references an application that does not exist.

14 Renewal & amendment

Closes G-20 · MOBILE FE P2

WHY

Both lines carry New / Renewal / Amendment as a type, and every permit except the Certificate of Occupancy expires — six or twelve months. The app files everything as New. An applicant with an expiring Fencing Permit has no renewal path, and one correcting a filed application has no amendment path.

BUILD

- A renewal flow that starts from an existing issued permit and carries forward what does not change.
- An amendment flow that references the application being amended.
- Prompt renewal from the expiry warning built in TAB 09.

ACCEPTANCE

- ☐ A renewal files with action Renewal and references the prior permit.
- ☐ An amendment references the amended application.
- ☐ Renewal is reachable from the expiring permit, not only from the catalog.

15 Closing certification

Verifies TABs 00–14 · PROGRAMME

BUILD

- Re-run the full comparison that produced this document, against the admin line as it stands on the day, and record what has drifted since 25 August.
- A standing parity test: permit types, document statuses, payment statuses and assessment statuses each asserted against a fixture taken from the admin source, so the next divergence fails a test rather than reaching an applicant.
- A closing verdict — CERTIFIED or NOT CERTIFIED — with every open gap named, its reason, and whether it is front-end, backend or a decision.

ACCEPTANCE

- ☐ Every G-## is closed, or open with a stated reason and an owner.
- ☐ The parity fixtures are dated and their source commit recorded.
- ☐ Backend-dependent items are recorded for the backend lane, not left implied.

4 · Cross-cutting requirements

These hold for every TAB. They are drawn from defects this codebase has already produced, not from general principle.

X1 · One definition, imported everywhere

This app has repeatedly grown a shared widget and then not reached for it: a wizard header copied sixteen times, a status badge hand-rolled five times, a responsive grid adopted by one screen of two, a confirmation screen copied fifteen times. Every TAB that introduces a shape used more than once must put it in one place and add an architecture test that fails on a copy.

X2 · Accessibility is not a later pass

Every new screen enters the existing suites: 100%, 200% text scale, 320dp, the 48dp touch floor, and the clipping check. The app clamps text scale at 2.0 precisely because that is the ceiling everything is tested at; a new screen that fails at 2.0 lowers what the app can honestly offer.

X3 · Failure is a state, not an exception

Every read shows a distinct failure state rather than an empty one — “you have no documents” is a claim about the applicant’s records and must not be made because a request timed out. Every write reports its failure, leaves the applicant’s work intact, and offers a retry. Every `await` followed by `setState` checks `mounted` first.

X4 · Say what is provisional

The admin’s catalog distinguishes national-law-verified requirements from Castilla-official-form-verified ones from pending placeholders. Carry that distinction to the applicant. A requirement the LGU has not confirmed must not be presented in the same voice as PD 1096.

X5 · Record the backend hand-offs

Several gaps cannot close front-end-only: contact verification needs a sender and a checker; official receipts, rejection reasons and per-document statuses need server fields to carry them. Build the states and the screens, drive them from the mock, and record each server dependency explicitly. Do not mock a thing into looking finished.

5 · Sources

Artefact	Where
Admin permit types, release model	<code>core/domain/permit.model.ts</code>
Requirements catalog (894 lines, 19 entries)	<code>core/domain/requirements-catalog.ts</code>
Lifecycle, evaluation, payment statuses	<code>core/domain/status.model.ts</code>
Document statuses & history	<code>core/domain/document.model.ts</code>
Assessment versioning & line items	<code>core/domain/assessment.model.ts</code>
Payment transactions, receipts, adjustments	<code>core/domain/payment.model.ts</code>
Applicant contact verification	<code>core/domain/applicant.model.ts</code>
Official form & checklist PDFs	<code>core/domain/permit-form-templates.ts</code>
Admin routes & pages	<code>src/app/app.routes.ts</code> , <code>src/app/pages/</code>
Mobile line compared	<code>eBPC0-Mobile-App</code> at <code>f603f53</code>

Philippine basis carried through. PD 1096 (National Building Code), RA 11032 (Ease of Doing Business — 3/7/20 working days, Citizen's Charter, zero contact), RA 9514 (Fire Code — FSEC/FSIC), Amended JMC 2021-01 (unified forms, joint inspections), RA 10173 (Data Privacy), RA 8792 (E-Commerce Act — functional equivalence, notarised documents excluded). Castilla-specific requirements come from the LGU's own forms as transcribed in the admin catalog, and carry that catalog's verification status.